

Location: Quake -> quantum-rings -> quantathon-harness

Input:

```
python - <<'PY'
import json
import numpy as np
from sklearn.model_selection import GroupKFold

from ml_common import (
    load_dataset,
    make_model,
    competition_score,
    CAP_SECONDS,
    MODEL_NAMES,
)

data, X, y_log, y_seconds, groups, feature_names = load_dataset()
thresholds = [16, 64, 512]

splitter = GroupKFold(n_splits=5)

all_oof = {}

print("=" * 70)
print("RUNNING ALL 9 INDIVIDUAL MODELS")
print("=" * 70)

for model_name in MODEL_NAMES:
    print()
    print(model_name.upper())
    print("-" * 50)

    oof_log = np.full(len(data), np.nan)

    for fold, (train_idx, test_idx) in enumerate(
        splitter.split(X, y_log, groups=groups),
        start=1,
```

```

):
    model = make_model(model_name)
    model.fit(X[train_idx], y_log[train_idx])

    pred_log = model.predict(X[test_idx])
    oof_log[test_idx] = pred_log

    pred_seconds_fold = np.clip(
        10 ** pred_log,
        1e-9,
        CAP_SECONDS,
    )

    fold_score = competition_score(
        y_seconds[test_idx],
        pred_seconds_fold,
    )

    print(f"Fold {fold}: {fold_score * 100:.2f}%")

pred_seconds = np.clip(
    10 ** oof_log,
    1e-9,
    CAP_SECONDS,
)

overall = competition_score(
    y_seconds,
    pred_seconds,
)

print(f"\nOverall OOF: {overall * 100:.2f}%")

for threshold in thresholds:
    mask = data["threshold"].to_numpy() == threshold

    score = competition_score(

```

```

        y_seconds[mask],
        pred_seconds[mask],
    )

    print(
        f"Threshold {threshold}: "
        f"{score * 100:.2f}%"
    )

all_oof[model_name] = oof_log

```

```

print()
print("=" * 70)
print("FINAL ENSEMBLE")
print("=" * 70)

```

```

with open("ensemble_choice.json", "r") as f:
    ensemble = json.load(f)

```

```

ensemble_log = np.zeros(len(data), dtype=float)

```

```

for model_name, weight in zip(
    ensemble["models"],
    ensemble["weights"],
):
    ensemble_log += weight * all_oof[model_name]

```

```

ensemble_seconds = np.clip(
    10 ** ensemble_log,
    1e-9,
    CAP_SECONDS,
)

```

```

overall = competition_score(
    y_seconds,
    ensemble_seconds,
)

```

)

```
print("\nWeights:")
for model_name, weight in zip(
    ensemble["models"],
    ensemble["weights"],
):
    print(f"{model_name}: {weight:.2f}")

print(f"\nOverall Ensemble OOF: {overall * 100:.2f}%")

for threshold in thresholds:
    mask = data["threshold"].to_numpy() == threshold

    score = competition_score(
        y_seconds[mask],
        ensemble_seconds[mask],
    )

    print(
        f"Threshold {threshold}: "
        f"{score * 100:.2f}%"
    )

print()
print("=" * 70)
print("DONE")
print("=" * 70)
PY
```

OUTPUT:

=====

RUNNING ALL 9 INDIVIDUAL MODELS

=====

EXTRA_TREES

Fold 1: 91.28%

Fold 2: 90.13%

Fold 3: 87.45%

Fold 4: 87.12%

Fold 5: 89.43%

Overall OOF: 89.08%

Threshold 16: 90.51%

Threshold 64: 88.90%

Threshold 512: 87.55%

RANDOM_FOREST

Fold 1: 89.25%

Fold 2: 88.16%

Fold 3: 85.56%

Fold 4: 85.21%

Fold 5: 87.78%

Overall OOF: 87.19%

Threshold 16: 88.78%

Threshold 64: 87.29%

Threshold 512: 85.12%

HIST_GRADIENT_BOOSTING

Fold 1: 90.85%

Fold 2: 89.06%

Fold 3: 87.25%

Fold 4: 87.15%
Fold 5: 88.70%

Overall OOF: 88.60%
Threshold 16: 90.00%
Threshold 64: 88.53%
Threshold 512: 86.98%

GRADIENT_BOOSTING

Fold 1: 86.35%
Fold 2: 85.96%
Fold 3: 84.85%
Fold 4: 82.60%
Fold 5: 84.63%

Overall OOF: 84.88%
Threshold 16: 87.04%
Threshold 64: 84.58%
Threshold 512: 82.59%

XGBOOST

Fold 1: 90.12%
Fold 2: 88.84%
Fold 3: 87.24%
Fold 4: 87.03%
Fold 5: 88.47%

Overall OOF: 88.34%
Threshold 16: 89.56%
Threshold 64: 88.22%
Threshold 512: 86.99%

LIGHTGBM

Fold 1: 90.99%

Fold 2: 89.26%
Fold 3: 87.04%
Fold 4: 86.63%
Fold 5: 89.46%

Overall OOF: 88.67%
Threshold 16: 90.29%
Threshold 64: 88.73%
Threshold 512: 86.63%

SVR

Fold 1: 81.86%
Fold 2: 82.26%
Fold 3: 80.23%
Fold 4: 80.86%
Fold 5: 79.47%

Overall OOF: 80.93%
Threshold 16: 83.61%
Threshold 64: 81.42%
Threshold 512: 77.06%

KNN

Fold 1: 85.36%
Fold 2: 84.93%
Fold 3: 82.13%
Fold 4: 81.72%
Fold 5: 83.24%

Overall OOF: 83.47%
Threshold 16: 85.56%
Threshold 64: 85.66%
Threshold 512: 78.26%

MLP

Fold 1: 85.56%
Fold 2: 82.81%
Fold 3: 81.75%
Fold 4: 82.04%
Fold 5: 84.96%

Overall OOF: 83.43%
Threshold 16: 84.07%
Threshold 64: 83.39%
Threshold 512: 82.67%

=====

FINAL ENSEMBLE

=====

Weights:
extra_trees: 0.70
lightgbm: 0.20
hist_gradient_boosting: 0.10

Overall Ensemble OOF: 89.21%
Threshold 16: 90.67%
Threshold 64: 89.04%
Threshold 512: 87.63%

=====

DONE

=====